Rapport technique

Projet ML complet - Analyse de sentiment YouTube

YouTube Sentiment Lens

Du développement au déploiement cloud

Filière INDIA | Année universitaire 2026/27
Université Mohamed V - ENSAM Rabat
Prof. Maria Zemzami

01 | Synthèse du projet

Objectif

Le projet automatise l'analyse de commentaires YouTube en trois classes : négatif (-1), neutre (0) et positif (1).

Solution

Une extension Chrome extrait les commentaires visibles, les transmet par batch à une API FastAPI et affiche le sentiment, la confiance et les statistiques.

Résultat

Le modèle atteint 87,79 % d'accuracy et dépasse les seuils F1 demandés. L'API est exécutée dans Docker sur GitHub Codespaces.

02 | Problème et architecture

Problématique

L'analyse manuelle de centaines de commentaires est lente, subjective et difficile à quantifier. Le système propose une analyse automatique et reproductible.

Architecture

Frontend : extension Chrome Manifest V3. Backend : FastAPI avec Pydantic et CORS. ML Engine : TF-IDF + Logistic Regression. Cloud : Docker dans Codespaces, port public 7860.

Flux

YouTube -> extraction -> POST /predict_batch -> modèle ML -> prédictions, confiance et statistiques.

03 | Données et préparation

Source

Le dataset Reddit Sentiment Analysis est téléchargé automatiquement depuis GitHub et sauvegardé dans data/raw/reddit.csv.

Nettoyage

URLs, mentions et hashtags sont remplacés par des tokens; les caractères spéciaux sont normalisés; les textes vides et doublons sont supprimés.

Statistiques

37 149 commentaires téléchargés; 36 562 conservés après nettoyage; 8 277 négatifs, 13 042 neutres et 15 830 positifs.

Reproductibilité

Split stratifié 80/20 avec random_state=42.

04 | Modélisation et optimisation

Vectorisation

TF-IDF valorise les termes discriminants et réduit l'influence des mots trop fréquents.

Algorithmes

Logistic Regression pondérée par classe comparée à Linear SVM. La Logistic Regression est retenue pour ses probabilités de confiance.

Optimisation

GridSearchCV teste C, min_df et ngram_range. Meilleurs paramètres : C=2.0, min_df=2, ngram_range=(1,1).

Artefacts

Pipeline, vectoriseur et classifieur sauvegardés avec joblib dans models/.

05 | Performance du modèle

Test set : 7 313 commentaires

Mesure	Résultat	Seuil	Statut
Accuracy	87,79 %	80 %	Atteint
F1 négatif	81,18 %	75 %	Atteint
F1 neutre	90,79 %	75 %	Atteint
F1 positif	88,69 %	75 %	Atteint
Inférence batch 50	3,57 ms	100 ms	Atteint

Matrice de confusion : [1318,149,180] / [60,2420,41] / [222,241,2682]. Les confusions principales concernent les commentaires ambigus.

06 | API REST et extension Chrome

API

GET /health vérifie le service. POST /predict_batch accepte 1 à 500 commentaires et renvoie label, sentiment, confiance, comptes et latence.

Extension

Extraction des commentaires visibles, statistiques en pourcentage, filtres, mode clair/sombre et copie des résultats.

Validation

Swagger /docs et OpenAPI vérifiés sur l'URL publique. Le flux réel YouTube -> extension -> API a été testé avec 19 commentaires.

07 | Déploiement, tests et livrables

Docker et Codespaces

Image python:3.10-slim, requirements-prod.txt, code et modèles. Commandes : docker build -t youtube-sentiment-api . puis docker run -d --name youtube-sentiment-api -p 7860:7860 -e PORT=7860 youtube-sentiment-api.

URL publique

https://curly-happiness-7vpr5wr6jgq9fpwjr-7860.app.github.dev. Codespaces est éphémère et doit rester actif pendant la démonstration.

Tests

3 tests automatisés passent. Docker build, health, OpenAPI, prédiction publique, syntaxe JavaScript et contrôles Playwright sont validés.

Livrables et conclusion

Dépôt GitHub, modèles joblib, rapport PDF, captures et chrome-extension.zip. Les critères ML et les fonctionnalités demandées sont respectés.

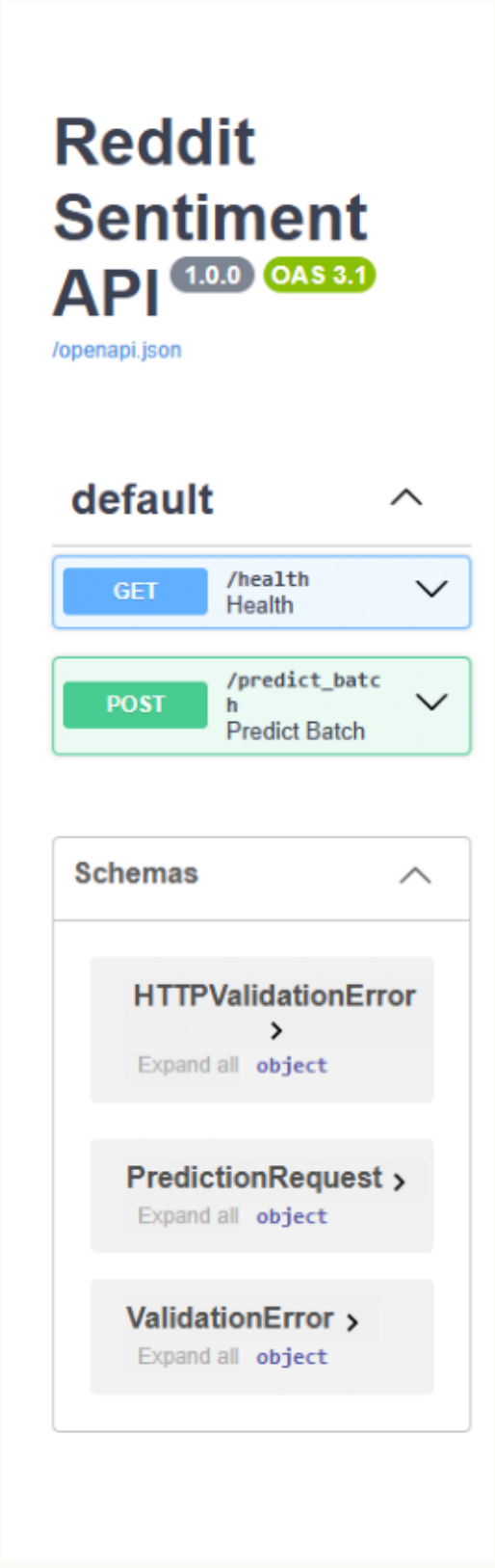
08 | Preuve API : health

Preuve capturée avec Playwright



09 | Preuve API : Swagger

Preuve capturée avec Playwright



10 | Preuve extension : mode clair

Preuve capturée avec Playwright



11 | Preuve extension : mode sombre

Preuve capturée avec Playwright

